

Partner Data API

<https://sandbox.octaneobit.com/orb/api/octanev4/partner>

Base URL: <https://sandbox.octaneobit.com/orb/api/octanev4/partner>

Read-only endpoints unless otherwise noted. All JSON. Results are automatically scoped to your account — you don't need to pass a site ID.

1. Authentication

Send your token in the Authorization header on every request:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Watch the spacing. Exactly one space after `Bearer`. Two spaces breaks it silently.

2. Read endpoints (GET)

Endpoint	What it returns	Optional filters
<code>/fuelstations</code>	Your fuel stations	none
<code>/pumps</code>	Pumps, with nozzles included	<code>fuelStationId</code>
<code>/fuelsales</code>	Fuel sale transactions	<code>fuelStationId</code> , <code>pump</code> , <code>pumpNozzleId</code>
<code>/merchants</code>	Merchant / company records	none

All filter values must be base64-encoded — see §3. All GET endpoints also accept the pagination/sorting params in §5 and share the response envelope and error shapes in §6.

2.1 GET /fuelstations

```
curl "https://sandbox.octaneobit.com/orb/api/octanev4/partner/fuelstations" \
-H "Authorization: Bearer YOUR_TOKEN_HERE"
```

No filters — returns every station scoped to your account.

Sample response

```
{
  "status": "success",
  "message": "Fuel stations data retrieved",
  "data": [
    {
      "record_id": "STN00001",
      "station_name": "Example Fuel Station",
      "station_code": "EX-001",
      "country": "Kenya",
      "county": "Example County",
      "region": "Example Region",
      "location": "Example Town, along Example Road",
      "contact_number": "+254700000000",
      "email": "station@example.com",
      "manager_name": "John Doe",
      "status": "active",
      "latitude": "-1.00000",
      "longitude": "37.00000",
      "google_maps_link": "https://maps.google.com/?q=-1.00000,37.00000",
      "opening_hours": "06:00-22:00",
      "is_operational": 1,
      "has_atg_system": 1,
      "licence_number": "LIC-0000-00000",
    }
  ]
}
```

```

    "licence_expiry_date": "2027-03-31",
    "erp_integration_code": "ERP-EX001",
    "till_number": "000000",
    "paybill_number": "000000",
    "bank_account_name": "Example Fuels Ltd",
    "bank_account_number": "0000000000",
    "default_payment_mode": "M-Pesa",
    "storage_capacity_litres": 40000,
    "additional_details": null,
    "created_on": "2026-06-30 08:28:03"
  }
],
"pagination": {
  "total_records": 6,
  "page_count": 1,
  "current_page": 1,
  "page_size": 20,
  "first_row": 1,
  "last_row": 6,
  "has_next": false,
  "has_prev": false
}
}

```

Field reference

Field	Description
<code>record_id</code>	Unique station identifier. Use this value for <code>fuelStationId</code> filters on other endpoints
<code>station_name</code> / <code>station_code</code>	Display name and short code
<code>country</code> / <code>county</code> / <code>region</code> / <code>location</code>	Geographic details
<code>contact_number</code> / <code>email</code> / <code>manager_name</code>	Station contact info
<code>status</code>	<code>active</code> / presumably <code>inactive</code> — confirm full enum
<code>latitude</code> / <code>longitude</code> / <code>google_maps_link</code>	Map coordinates
<code>opening_hours</code>	Free-text operating hours
<code>is_operational</code>	<code>1</code> / <code>0</code> — whether the station is currently trading
<code>has_atg_system</code>	<code>1</code> / <code>0</code> — whether an Automatic Tank Gauge is installed
<code>licence_number</code> / <code>licence_expiry_date</code>	Regulatory licence details
<code>erp_integration_code</code>	Code used to match this station in your own ERP, if applicable
<code>till_number</code> / <code>paybill_number</code>	M-Pesa collection details
<code>bank_account_name</code> / <code>bank_account_number</code>	Settlement bank account
<code>default_payment_mode</code>	Default payment method recorded at this station
<code>storage_capacity_litres</code>	Total tank storage capacity
<code>additional_details</code>	Free-text notes, often <code>null</code>
<code>created_on</code>	Record creation timestamp

2.2 GET /pumps

```

curl "https://sandbox.octaneobit.com/orb/api/octanev4/partner/pumps?fuelStationId=BASE64_VALUE" \
-H "Authorization: Bearer YOUR_TOKEN_HERE"

```

Optional: `?fuelStationId=` to filter to one station (base64-encoded, see §3).

Sample response

```

{
  "status": "success",
  "message": "Fuel pumps data retrieved",
  "data": [

```

```
{
  "record_id": "PMP00006",
  "fuel_station_id": "STN00001",
  "pump_id": "6",
  "pump_name": "6",
  "manufacturer": "",
  "model_number": "",
  "installation_date": "2026-06-30",
  "status": "active",
  "created_on": "2026-06-30 08:28:03",
  "_fuel_stations_station_name_fuel_station_id": "Example Fuel Station",
  "nozzles": [
    {
      "pump_id": "PMP00006",
      "nozzle_label": "1",
      "fuel_type": "VPOWER",
      "tank_reference": "TNK00001"
    },
    {
      "pump_id": "PMP00006",
      "nozzle_label": "3",
      "fuel_type": "DIESEL",
      "tank_reference": "TNK00001"
    }
  ]
},
{
  "pagination": {
    "total_records": 6,
    "page_count": 1,
    "current_page": 1,
    "page_size": 20,
    "first_row": 1,
    "last_row": 6,
    "has_next": false,
    "has_prev": false
  }
}
```

Field reference — pump record

Field	Description
<code>record_id</code>	Unique pump identifier. Use this value (not <code>pump_id</code>) when filtering <code>/fuelsales?pump=</code>
<code>fuel_station_id</code>	The station this pump belongs to
<code>pump_id</code>	Human-readable pump number as displayed at the station (e.g. <code>"1"</code> - <code>"6"</code>) — for display only
<code>pump_name</code>	Display label, currently mirrors <code>pump_id</code>
<code>manufacturer</code> / <code>model_number</code>	Equipment details (often blank in sandbox data)
<code>installation_date</code>	Date the pump was installed
<code>status</code>	<code>active</code> / presumably <code>inactive</code> — confirm full enum
<code>created_on</code>	Record creation timestamp
<code>_fuel_stations_station_name_fuel_station_id</code>	Joined station name, for display convenience
<code>nozzles[]</code>	Array of nozzles attached to this pump (max 10)

Field reference — nozzle object (nested in `nozzles[]`)

Field	Description
<code>pump_id</code>	⚠ Despite the name, this holds the parent pump's <code>record_id</code> (e.g. <code>PMP00006</code>), not the display <code>pump_id</code> number. Don't confuse the two
<code>nozzle_label</code>	Nozzle number on the pump (e.g. <code>"1"</code> , <code>"2"</code> , <code>"3"</code>)

<code>fuel_type</code>	Fuel grade dispensed by this nozzle (<code>VPOWER</code> , <code>DIESEL</code> , <code>UNLEADED</code> , etc.)
<code>tank_reference</code>	Underground tank this nozzle draws from

2.3 GET /fuelsales

```
curl "https://sandbox.octaneobit.com/orb/api/octanev4/partner/fuelsales?fuelStationId=BASE64_VALUE" \
-H "Authorization: Bearer YOUR_TOKEN_HERE"
```

Optional: `?fuelStationId=` , `&pump=` , `&pumpNozzleId=` (each value base64-encoded, see §3). `pump` and `pumpNozzleId` take the `record_id` values from `/pumps` , not the display pump number.

Sample response

```
{
  "status": "success",
  "message": "Fuel sales data retrieved",
  "data": [
    {
      "primkey": 10482,
      "record_id": "FS-20260830-0001",
      "sale_date": "2026-08-30T14:22:10.000Z",
      "pump": "PMP00003",
      "pump_nozzle_id": "NOZ00003A",
      "fuel_type": "Diesel",
      "quantity_sold_litres": 45.75,
      "sale_price_per_litre": 182.50,
      "total_amount": 8349.38,
      "total_paid": 8349.38,
      "kra_id": "KRA-INV-00001",
      "sold_by_staff_id": "STF00001",
      "shift_id": "SHF00001",
      "fuel_station_id": "STN00001",
      "sale_method": "M-Pesa",
      "vehicle_plate": "KAA 000A",
      "customer_id": "CMP00001",
      "_fuel_pumps_pump_name_pump": "Pump 3",
      "_fuel_pump_nozzles_nozzle_label_pump_nozzle_id": "Nozzle A",
      "_system_users_name_sold_by_staff_id": "John Doe",
      "_shifts_shift_name_shift_id": "Morning Shift",
      "_fuel_stations_station_name_fuel_station_id": "Example Fuel Station",
      "_clients_client_name_customer_id": "Example Corp"
    },
    {
      "primkey": 10481,
      "record_id": "FS-20260830-0000",
      "sale_date": "2026-08-30T13:58:47.000Z",
      "pump": "PMP00001",
      "pump_nozzle_id": "NOZ00001B",
      "fuel_type": "Petrol",
      "quantity_sold_litres": 20.00,
      "sale_price_per_litre": 194.20,
      "total_amount": 3884.00,
      "total_paid": 3884.00,
      "kra_id": "KRA-INV-00002",
      "sold_by_staff_id": "STF00002",
      "shift_id": "SHF00001",
      "fuel_station_id": "STN00001",
      "sale_method": "Cash",
      "vehicle_plate": null,
      "customer_id": null,
      "_fuel_pumps_pump_name_pump": "Pump 1",
      "_fuel_pump_nozzles_nozzle_label_pump_nozzle_id": "Nozzle B",
      "_system_users_name_sold_by_staff_id": "Jane Doe",
      "_shifts_shift_name_shift_id": "Morning Shift",
      "_fuel_stations_station_name_fuel_station_id": "Example Fuel Station",
      "_clients_client_name_customer_id": null
    }
  ],
  "pagination": {
    "total_records": 4820,

```

```

    "page_count": 2410,
    "current_page": 1,
    "page_size": 2,
    "first_row": 1,
    "last_row": 2,
    "has_next": true,
    "has_prev": false
  }
}

```

Field reference

Field	Description
<code>record_id</code>	Unique transaction identifier. Use this as <code>fuelSaleId</code> when posting to the IPN listener (\$7)
<code>sale_date</code>	Timestamp the sale was recorded
<code>pump</code>	Pump's <code>record_id</code> (see <code>/pumps</code>), not the display pump number
<code>pump_nozzle_id</code>	Nozzle's <code>record_id</code> (see <code>nozzles[]</code> in <code>/pumps</code>)
<code>fuel_type</code>	Fuel grade dispensed (e.g. <code>Diesel</code> , <code>Petrol</code>)
<code>quantity_sold_litres</code>	Volume dispensed
<code>sale_price_per_litre</code>	Unit price at time of sale
<code>total_amount</code>	<code>quantity_sold_litres × sale_price_per_litre</code>
<code>total_paid</code>	Amount actually paid — compare against <code>total_amount</code> to detect under/over payment
<code>kra_id</code>	KRA e-tims invoice reference, if issued
<code>sold_by_staff_id</code>	Attendant who processed the sale
<code>shift_id</code>	Shift the sale falls under
<code>fuel_station_id</code>	Station where the sale occurred
<code>sale_method</code>	Payment method (e.g. <code>Cash</code> , <code>M-Pesa</code> , <code>Card</code>)
<code>vehicle_plate</code>	Vehicle plate, if captured — <code>null</code> when not recorded
<code>customer_id</code>	Linked client/customer record, if any — <code>null</code> for walk-in sales
<code>_fuel_pumps_pump_name_pump</code>	Joined pump display name, for convenience
<code>_fuel_pump_nozzles_nozzle_label_pump_nozzle_id</code>	Joined nozzle label
<code>_system_users_name_sold_by_staff_id</code>	Joined staff name
<code>_shifts_shift_name_shift_id</code>	Joined shift name
<code>_fuel_stations_station_name_fuel_station_id</code>	Joined station name
<code>_clients_client_name_customer_id</code>	Joined client/company name, if <code>customer_id</code> is set

An empty result set (e.g. filtered to a station/pump with no matching sales) still returns `status: "success"` with `"data": []` and a `pagination` block showing `"total_records": 0`.

2.4 GET /merchants

```

curl "https://sandbox.octaneobit.com/orb/api/octanev4/partner/merchants" \
-H "Authorization: Bearer YOUR_TOKEN_HERE"

```

No filters — returns every merchant/company record scoped to your account.

Sample response

```

{
  "status": "success",
  "message": "Merchants data retrieved",
  "data": [

```

```
{
  "company_id": "CMP00001",
  "business_no": "BN-0000-0001",
  "name": "John Doe",
  "mobile": "+254700000000",
  "email": "john.doe@example.com",
  "business_name": "Example Corp",
  "location": "Example Business District",
  "specialty": "Example Industry",
  "remark": null,
  "logo": "https://cdn.example.com/logos/example-corp.png",
  "bank_name": "Example Bank",
  "account_no": "0000000000000",
  "pin_no": "P000000000X",
  "swort_code": "00000"
},
{
  "pagination": {
    "total_records": 1,
    "page_count": 1,
    "current_page": 1,
    "page_size": 20,
    "first_row": 1,
    "last_row": 1,
    "has_next": false,
    "has_prev": false
  }
}
```

Field reference

Field	Description
<code>company_id</code>	Unique merchant identifier. Matches <code>customer_id</code> on <code>/fuelsales</code> records for this merchant
<code>business_no</code>	Business registration number
<code>name</code>	Contact person's name
<code>mobile</code> / <code>email</code>	Contact details
<code>business_name</code>	Registered/trading name — corresponds to <code>_clients_client_name_customer_id</code> on <code>/fuelsales</code>
<code>location</code>	Business address
<code>specialty</code>	Business category/industry
<code>remark</code>	Free-text notes, often <code>null</code>
<code>logo</code>	URL to the merchant's logo image
<code>bank_name</code> / <code>account_no</code>	Settlement bank account
<code>pin_no</code>	KRA PIN
<code>swort_code</code>	Bank branch/swift-style sort code

3. Filter values must be base64-encoded

If you add any filter (like `fuelStationId=`), encode the value first. Plain text will not match.

```
# bash
echo -n "FS018" | base64
```

```
# python
base64.b64encode(b"FS018").decode()
```

```
// node
Buffer.from("FS018").toString("base64")
```

```
# powershell
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes("FS018"))
```

4. Response envelope (all GET endpoints)

Every GET endpoint returns the same shape — only `message` and the contents of `data` / `pagination` differ:

```
{
  "status": "success",
  "message": "...",
  "data": [ ],
  "pagination": {
    "total_records": 42,
    "page_count": 3,
    "current_page": 1,
    "page_size": 20,
    "first_row": 1,
    "last_row": 20,
    "has_next": true,
    "has_prev": false
  }
}
```

5. Pagination & sorting (optional, plain text — no encoding)

Param	Example
<code>pageNo</code>	<code>?pageNo=2</code>
<code>pageSize</code>	<code>?pageSize=50</code> (max 2000, default 20)
<code>orderBy</code>	<code>?orderBy=saleDate</code>
<code>orderType</code>	<code>?orderType=DESC</code>

6. Errors

Status	<code>status</code> value	Meaning
403	<code>unauthorized</code>	Bad, missing, or expired token
200	<code>error</code>	Token is valid but your role lacks permission for this endpoint's data — <code>data</code> is <code>[]</code>
500	<code>error</code>	Server error

Bad/expired token

```
{
  "status": "unauthorized",
  "message": "Invalid signature"
}
```

Valid token, insufficient role

```
{
  "status": "error",
  "message": "Access denied: role does not have permission to select fuel_sales",
  "data": [ ]
}
```

Server error

```
{
  "status": "error",
  "message": "Connection lost: The server closed the connection."
}
```

7. Posting payment confirmation — IPN Listener (POST)

Once a fuel transaction has been processed on your PDQ/payment terminal, post the result back to us here so we can mark the transaction as **PAID** on our side. This corresponds to steps 16–17 of the PayFuel payment flow (POS sends payment result → server updates fuel transaction as PAID).

Endpoint

POST <https://sandbox.octaneobit.com/orb/api/octanev4/ipnlistener>

Headers

Authorization: Bearer YOUR_TOKEN_HERE
Content-Type: application/json

Sample request body — DRAFT, TO BE CONFIRMED

The field names below are a proposed schema based on the payment flow diagram (transaction, pump/nozzle, amount, PDQ approval, status). They have **not** been verified against the live endpoint yet — confirm exact field names, types, and required/optional status internally before sharing this with Crystal Soft.

```
{
  "fuelSaleId": "TXN00123",
  "fuelStationId": "FS018",
  "pumpId": "PMP04",
  "pumpNozzleId": "NZ01",
  "amount": 4500.00,
  "currency": "KES",
  "paymentStatus": "APPROVED",
  "paymentMethod": "CARD",
  "pdqReference": "PDQ-REF-8891234",
  "approvalCode": "AUTH00456",
  "transactionDate": "2026-08-31T10:15:00Z"
}
```

Field	Description
<code>fuelSaleId</code>	The transaction ID returned by <code>/fuelsales</code> for the sale being paid
<code>fuelStationId</code>	Station identifier (plain text in this body — not base64, unlike GET filters)
<code>pumpId</code> / <code>pumpNozzleId</code>	Pump and nozzle that dispensed the fuel
<code>amount</code> / <code>currency</code>	Amount charged and currency
<code>paymentStatus</code>	e.g. <code>APPROVED</code> / <code>DECLINED</code> — confirm accepted enum values
<code>paymentMethod</code>	e.g. <code>CARD</code> , <code>MOBILE_WALLET</code> , <code>QR</code> — confirm accepted values
<code>pdqReference</code>	Reference/receipt number from the PDQ terminal
<code>approvalCode</code>	Bank/processor authorization code, if applicable
<code>transactionDate</code>	ISO 8601 timestamp of when payment was processed

Expected response

```
{
  "status": "success",
  "message": "Fuel transaction updated as PAID"
}
```


On failure, expect the same error shapes as §6 (403 for bad/expired token, 500 for server error) — confirm whether a 4xx is also returned for a `fuelSaleId` that doesn't exist or is already marked PAID.

Questions or need a token? Contact us directly — don't share tokens over email/chat that isn't secure.